

Restless Multi arm Bandits

Bypassing Scalar Rewards with Direct Index Policies

BetaStay

May 8, 2026

H-SRAC

H-SRAC: System Architecture Overview

The Homogeneous State-Ranking Actor-Critic (H-SRAC) algorithm consists of four synergistic components that bridge Deep RL with Whittle Index theory:

- **The Global Actor (π_θ):** A "Universal Ranker" that maps the entire state space to priority scores (Logits), ensuring homogeneous treatment across all arms.
- **The Newton-Balanced Sigmoid:** A differentiable "Budget Regulator" that shifts Actor scores using a global bias b to ensure exactly M arms are picked on average.
- **The State-Specific Critic (V_ϕ):** A "Value Estimator" that evaluates individual state quality to generate the Advantage (δ) signal for training.
- **Stochastic Sensitivity (λ):** A "Sharpness Control" sampled during training to force the Actor to learn robust rankings regardless of policy entropy.

H-SRAC: Detailed Mathematical Architecture

1. Actor Forward Pass: Universal State Ranking

The Actor π_θ serves as a **Global Ranker**. It maps the constant state-space vector $\mathbf{S}_{all}$ to a priority logit vector $\mathbf{H}$:

$$\mathbf{H} = [h_1, h_2, \dots, h_S] = \pi_\theta([s_1, s_2, \dots, s_S])$$

State Retrieval:

For any physical arm i at time t , we retrieve the logit corresponding to its specific state $s(i)^t \in \mathcal{S}$:

$$h(s(i)^t) = \text{Lookup}(\mathbf{H}, s(i)^t)$$

Identical states across different machines receive identical priority scores.

2. Budget-Balanced Probability Mapping

We transform logits into selection probabilities $p_i^t \in [0, 1]$ while enforcing the budget M :

$$p_i^t = \sigma(\lambda h(s(i)^t) + b) = \frac{1}{1 + \exp(-(\lambda h(s(i)^t) + b))}$$

Solving for Global Bias b (Newton's Method):

We find the unique b such that the expected number of active arms is M :

$$f(b) = \sum_{i=1}^N p_i^t - M = 0$$

Iterative Update:

$$b_{next} = b - \frac{f(b)}{f'(b)} = b - \frac{\sum p_i^t - M}{\sum p_i^t(1 - p_i^t)}$$

3. H-SRAC: Proof of Expected Budget Constraint

We want to show that the expected number of active arms equals the budget M :

$$E \left[\sum_{i=1}^N a_i^t \right] = M$$

Proof:

1. By the **Linearity of Expectation**:

$$E \left[\sum_{i=1}^N a_i^t \right] = \sum_{i=1}^N E[a_i^t]$$

2. Since a_i^t is a Bernoulli random variable with probability p_i^t :

$$E[a_i^t] = P(a_i^t = 1) \cdot 1 + P(a_i^t = 0) \cdot 0 = p_i^t$$

3. Our Newton Solver specifically finds the bias b such that:

$$\sum_{i=1}^N p_i^t = \sum_{i=1}^N \sigma(\lambda h(s(i)^t) + b) = M$$

4. Substituting (2) and (3) into (1):

$$\sum_{i=1}^N E[a_i^t] = \sum_{i=1}^N p_i^t = M \quad \blacksquare$$

This ensures the global constraint is satisfied in expectation for every timestep t .

4. The Critic and Advantage Signal

The Critic V_ϕ estimates the value of a **single state**. We evaluate it for all N arms to compute the **Advantage Function** δ_i^t :

Temporal Difference (TD) Error:

$$\delta_i^t = \underbrace{r_i^t + \gamma V_\phi(s(i)^{t+1})}_{\text{Target}} - \underbrace{V_\phi(s(i)^t)}_{\text{Estimate}}$$

Interpretation:

- δ_i^t is the "error signal" indicating if an action was better than average.
- Large positive δ_i^t occurs when an arm avoids a deadline penalty F by being prioritized.

5. Dual-Network Parameter Updates

Critic Update (Value Refinement):

Minimize the Mean Squared Error of the TD-target:

$$\phi \leftarrow \phi + \alpha_\phi \sum_{i=1}^N \delta_i^t \nabla_\phi V_\phi(s(i)^t)$$

Actor Update (Ranking Refinement):

Update weights θ using the individual advantages via the **Straight-Through Estimator (STE)**:

$$\theta \leftarrow \theta + \alpha_\theta \sum_{i=1}^N \delta_i^t \nabla_\theta \log \pi_\theta(a_i^t \mid s(i)^t, \lambda)$$

$$\nabla_\theta \log \pi_\theta \approx \nabla_\theta \log \sigma(\lambda h(s(i)^t) + b)$$

6. Stochastic Sensitivity (λ) and Exploration

We sample $\lambda \sim \mathcal{U}(1, 20)$ per episode to vary policy sharpness.

The Exploration Trade-off:

- **Low λ (Soft):** Probabilities are flattened. This forces the network to explore the **Slack Zone** ($B < D$).
- **High λ (Hard):** Sigmoid approaches a step function. This simulates **Inference Time** (Top- M ranking).

This ensures the learned logits h_s converge to true **Whittle Index** values.

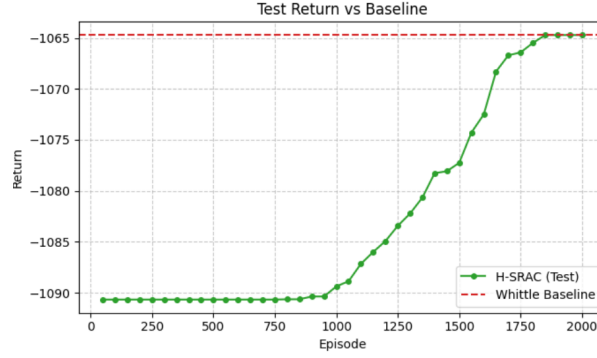
Problem Setting: Deadline Scheduling

Objective: Minimize total penalty from missed deadlines in an RMAB environment.

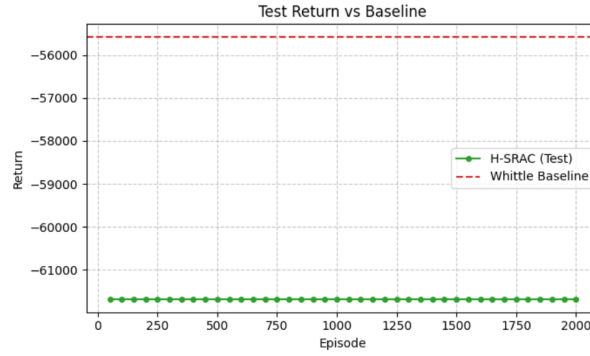
- **State** (B, D): B is remaining workload; D is time until deadline.
- **Active Transition:** Pulling an arm reduces workload B .
- **Passive Transition:** Ignoring an arm reduces deadline D .
- **Penalty:** If D reaches 1 and $B > 0$, a massive penalty is incurred.

□

Results: Test

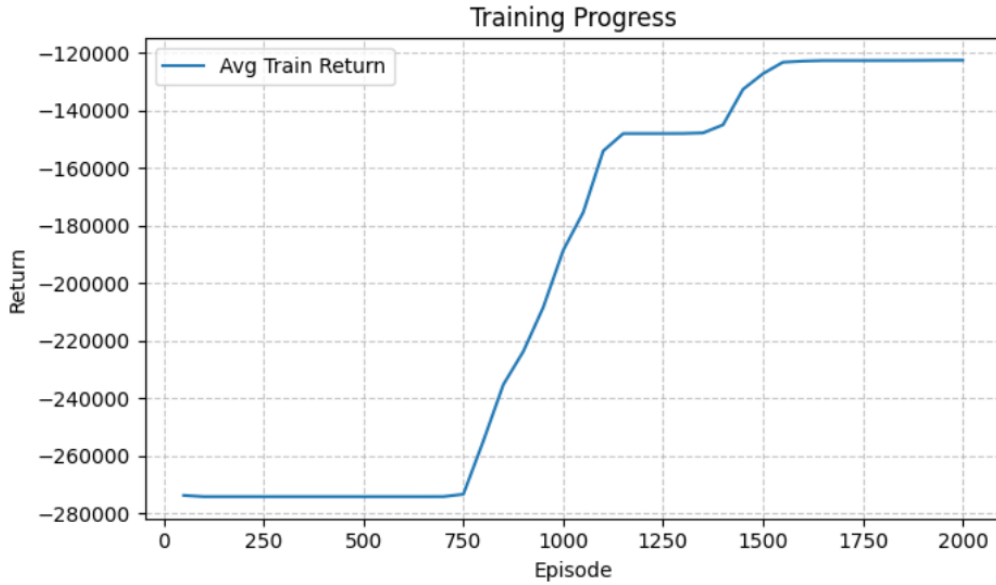


Test Reward ($N = 4, M = 1$)



Test Reward ($N = 100, M = 10$)

Results: Training



Training Return ($N = 100$, $M = 10$)

- **The Paradox:** Training return improved by $\approx 150,000$ points while Test Reward remained flat for 1600+ episodes.
- **Population Buffer:** The large population of passive states stabilizes the training curve compared to the $N = 4$ case.

Mechanism: Why $N = 100$ Converges Differently

- **Population Sensitivity ($N = 4$, $M = 1$):**
 - With only 4 total states to manage, the leaderboard is highly sensitive.
 - Any small change in the ranking of any subset of states results in an immediate change to the Top-1 selection.
- **The $N = 100$, $M = 10$ Buffer:**
 - Sorting 100 states is significantly more complex.
 - Picking the Top-10 requires many more state-rankings to shift before the "Elite" group is physically displaced, causing the flat test reward.
- **Stochastic Training vs. Deterministic Testing:**
 - **Training Loss:** Changes constantly due to the **stochastic nature** of the environment and sampling. Small probability shifts across the 100 arms reduce the total penalty sum.
 - **Test Reward:** Remains flat because it is a deterministic "Hard Top-M" pick that only budges when the absolute best states change.

BT-PPO-SRAC

Enhanced State-Ranking Actor-Critic

Addition 1

Pairwise Learning:

The Bradley-Terry Integration

- Replaces global bias gradients with head-to-head tournaments.

Addition 2

Stabilizing Rankings:

PPO Clipping

- Prevents violent logit fluctuations using a trust-region surrogate objective.

Core Thesis: Decoupling State-Hierarchy Learning from Budget Constraints.

Pairwise Learning: The Bradley-Terry Integration (BT-SRAC)

Objective: Shift from absolute logit regression to relative preference learning to stabilize state-ranking.

- **Bradley-Terry Model:** Probability that state s_i is ranked above s_j :

$$P(s_i \succ s_j) = \frac{\exp(\beta h(s_i))}{\exp(\beta h(s_i)) + \exp(\beta h(s_j))}$$

- **Actor Update Equation:** For a sampled pair (i, j) , update parameters θ via the gradient of the pairwise cross-entropy loss:

$$\nabla_{\theta} \mathcal{L} = \beta \cdot \underbrace{[P(s_i \succ s_j) - y_{ij}]}_{\text{Prediction Error}} \cdot \underbrace{(\nabla_{\theta} h(s_i) - \nabla_{\theta} h(s_j))}_{\text{Directional Shift}}$$

where $y_{ij} = \mathbb{I}[\delta_i > \delta_j]$ is the indicator of relative advantage.

- **Key Advantages:**

- **Scale Invariance:** Depends only on the sign of advantage differences.

Stabilizing Rankings with PPO Clipping (BT-PPO-SRAC)

- **The Problem:** Pure Bradley-Terry "tournaments" can trigger aggressive updates.
- **The Solution:** Integrate a PPO-style trust region to constrain how rapidly the relative preference between any two states can shift.

- **Mechanism:** Maintain a target anchor policy ($\pi_{\theta_{old}}$) to compute the **Ranking Probability Ratio**:

$$r_{ij}(\theta) = \frac{P_{\theta_{new}}(s_i \succ s_j)}{P_{\theta_{old}}(s_i \succ s_j)}$$

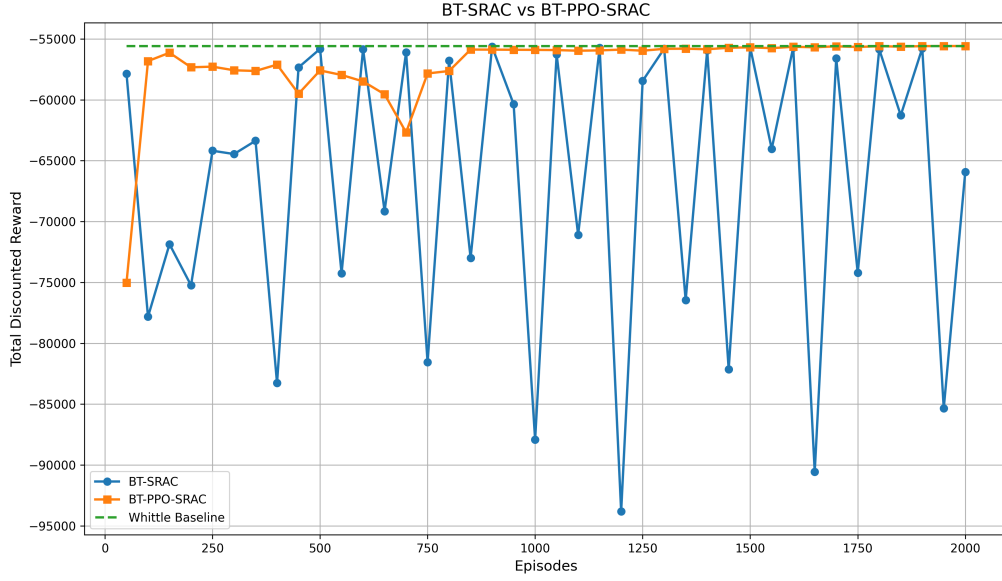
- **Clipped Actor Objective:** For a pair where the Critic identifies s_i as the winner ($y_{ij} = 1$), we minimize:

$$\mathcal{L}_{Actor} = -\mathbb{E}_{i,j} \left[\min \left(r_{ij}(\theta), \text{clip}(r_{ij}(\theta), 1 - \epsilon, 1 + \epsilon) \right) \right]$$

- **Impact on H-SRAC:**

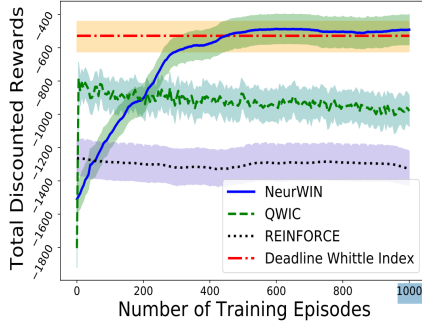
- **Trust Region:** Prevents one "unlucky" episode from tanking an elite state's priority.
- **Smooth Sorting:** Allows the 90 passive states to climb the ranking ladder steadily without destabilizing the current Top-10.

Results: Training

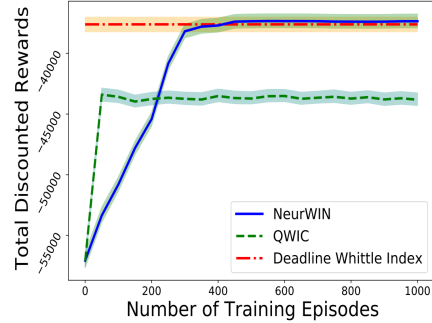


BT-SRAC vs BT-PPO-SRAC

Results: NEURWIN



(a) $N = 4$. $M = 1$.



(b) $N = 100$. $M = 10$.

Test Return for NEURWIN

- **Fast Convergence:** NEURWIN converges within 500 episodes for both settings ($N = 4, M = 1$ and $N = 100, M = 10$), demonstrating efficient learning and stable training behavior.

- **Comparison with H-SRAC:** H-SRAC converges for the smaller setting ($N = 4, M = 1$), but requires nearly 2000 episodes. For the larger setting ($N = 100, M = 10$), the test return shows minimal improvement, indicating poor scalability.
- **Comparison with BT-SRAC:** BT-SRAC performs reasonably for smaller settings, but in the larger setting ($N = 100, M = 10$) the test return fluctuates significantly and does not converge stably.
- **Comparison with BT-PPO-SRAC:** BT-PPO-SRAC eventually converges for the larger setting ($N = 100, M = 10$), but only after approximately 2000 episodes, making it substantially slower than NEURWIN.
- **Overall Observation:** Among all compared methods, NEURWIN achieves the fastest and most stable convergence across both small-scale and large-scale environments.

Proposed Approach: Ranking-based Actor–Critic Policy

Key Idea:

- Model RMAB scheduling as a **learned ranking problem**
- Instead of selecting actions directly, learn a **score per arm**
- Convert scores into a **top- M stochastic ranking policy**

Core Architecture:

- **Actor Network** $f_\theta(x_i^t)$:
- **Critic Network** $V_\psi(s_t)$:

Policy Construction:

- Scores $\rightarrow$ Plackett–Luce ranking distribution
- Sample top- M arms using Gumbel-top- M
- Produces stochastic but structured scheduling policy

Core Idea: Learn a Score per Arm

Step 1: Per-arm encoding

For each arm i with state x_i^t :

$$s_i^t = f_\theta(x_i^t)$$

Interpretation:

- s_i^t = “priority / urgency score”
- Computed using a shared neural network
- Same function applied to all arms

Output of actor:

$$\{s_1^t, s_2^t, \dots, s_N^t\}$$

Action Selection Problem

Goal:

- Select M arms out of N
- But selection must be:
 - stochastic (exploration)
 - differentiable (for learning)

Naive idea (fails):

- Take top- M scores:

$$\text{TopM}(s_1^t, \dots, s_N^t)$$

- Not differentiable $\Rightarrow$ no gradient flow

Solution idea:

- Convert scores into a probability distribution
- Sample M arms without replacement
- Use **Plackett-Luce ranking model**

Plackett-Luce Ranking Policy (Key Idea)

Step 2: Sampling procedure:

- Add Gumbel noise to scores

$$\tilde{s}_i = s_i + g_i, \quad g_i \sim \text{Gumbel}(0, 1)$$

- Sort perturbed scores
- Take top- M (our ranking now)

Convert scores into a ranking distribution

$$P(a_1, a_2, \dots, a_M \mid s) = \prod_{k=1}^M \frac{\exp(s_{a_k})}{\sum_{j \notin \{a_1, \dots, a_{k-1}\}} \exp(s_j)}$$

Gumbel-Max Trick and Plackett–Luce Sampling

Why does Gumbel work?
Gumbel noise is generated as

$$g_i = -\log(-\log u_i), \quad u_i \sim \text{Uniform}(0, 1)$$

Sampling rule

$$a = \arg \max_i (\tilde{s}_i) = \arg \max_i (s_i + g_i)$$

Probability of selecting arm i

$$P(a = i) = P(s_i + g_i \geq s_j + g_j, \forall j)$$

Rearranging,

$$P(a = i) = P(g_j \leq g_i + s_i - s_j, \forall j)$$

Using the Gumbel CDF,

$$F(g) = \exp(-e^{-g})$$

and independence of noises,

$$P(a = i \mid g_i) = \prod_{j \neq i} \exp\left(-e^{-(g_i + s_i - s_j)}\right)$$

$$= \exp\left(-e^{-g_i} \sum_{j \neq i} e^{s_j - s_i}\right)$$

Integrating over g_i yields

$$P(a = i) = \frac{\exp(s_i)}{\sum_j \exp(s_j)}$$

Conclusion

Adding i.i.d. Gumbel noise and taking the argmax is exactly equivalent to sampling from a softmax distribution. Sequential sampling without replacement gives the Plackett–Luce ranking model.

Why is

$$P(a_1, a_2, \dots, a_M \mid s)$$

a valid probability?

For each selection step k ,

$$P(a_k \mid a_1, \dots, a_{k-1}) = \frac{\exp(s_{a_k})}{\sum_{j \notin \{a_1, \dots, a_{k-1}\}} \exp(s_j)}$$

Since $\exp(s_j) > 0$ for all j ,

$$\sum_{j \notin \{a_1, \dots, a_{k-1}\}} \exp(s_j) \geq \exp(s_{a_k})$$

since the summation on LHS also contains $\exp(s_{a_k})$. Thus, denominator always greater than equal to numerator. hence it is a valid prob. distribution. Hence every conditional selection probability is valid, and the overall ranking probability

$$P(a_1, \dots, a_M \mid s) = \prod_{k=1}^M P(a_k \mid a_1, \dots, a_{k-1})$$

is also bounded in $[0, 1]$.

Derivation (Plackett–Luce):

Using Luce’s choice axiom, the probability of selecting arm i from a candidate set A is

$$P(i \mid A) = \frac{\exp(s_i)}{\sum_{j \in A} \exp(s_j)}$$

- Select a_1 from all arms

$$P(a_1) = \frac{\exp(s_{a_1})}{\sum_j \exp(s_j)}$$

- Remove a_1 , then select a_2

$$P(a_2 \mid a_1) = \frac{\exp(s_{a_2})}{\sum_{j \neq a_1} \exp(s_j)}$$

- Continue similarly up to a_M

Applying the chain rule:

$$P(a_1, \dots, a_M) = \prod_{k=1}^M P(a_k \mid a_1, \dots, a_{k-1})$$

which gives

$$P(a_1, a_2, \dots, a_M \mid s) = \prod_{k=1}^M \frac{\exp(s_{a_k})}{\sum_{j \notin \{a_1, \dots, a_{k-1}\}} \exp(s_j)}$$

Critic + Advantage for Ranking Policy

Critic (Value Function): Evaluates whether a sampled ranking is good or bad.

$$s_t := (x_1^t, x_2^t, \dots, x_N^t)$$

$$V(s_t) = \mathbb{E}[\text{return from } s_t]$$

One-step rollout target:

- At step t , actor produces a ranking and selects action
- Transition to next state s_{t+1}

$$y_t = r_t + \gamma V(s_{t+1})$$

Critic update (TD-style):

$$\mathcal{L}_{critic} = (V(s_t) - (r_t + \gamma V(s_{t+1})))^2$$

Advantage Signal for Ranking Policy

Advantage of sampled ranking:

$$A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Interpretation:

- $A_t > 0 \Rightarrow$ ranking better than expected
- $A_t < 0 \Rightarrow$ ranking worse than expected

Policy update (actor):

$$\nabla_{\theta} J(\theta) = \mathbb{E}[A_t \nabla_{\theta} \log P_{\theta}(a_{1:M} \mid s_t)]$$

Improve the policy parameters θ in the direction that increases the log-probability of good actions (rankings), scaled by how good they were (advantage).

Policy update (actor):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s_t \sim d^{\pi_{\theta}}, a_{1:M} \sim P_{\theta}(\cdot \mid s_t)} [A_t \nabla_{\theta} \log P_{\theta}(a_{1:M} \mid s_t)]$$

where:

- $d^{\pi_{\theta}}(s_t)$: state distribution induced by the current policy
- $P_{\theta}(a_{1:M} \mid s_t)$: probability of generating a ranked list (Plackett–Luce policy)
- $A_t = R_t - V_{\psi}(s_t)$: advantage function (baseline-corrected return)

Interpretation:

- The expectation is over trajectories induced by the policy: states s_t , sampled rankings $a_{1:M}$, and environment stochasticity.
- If $A_t > 0$, increase probability of the sampled ranking: $P_{\theta}(a_{1:M} \mid s_t)$.
- If $A_t < 0$, decrease its probability.
- This performs gradient ascent on expected return under the current policy.

Proximal Policy Optimization

Policy Gradient Foundation

$$\nabla_{\theta} J(\theta) = \mathbb{E} [A_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

Problem:

- No restriction on how much π_{θ} changes per update
- Can lead to large policy shifts and collapse

PPO Objective (How it fixes the problem)

Step 1: Probability ratio

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

- $r_t = 1$: no change
- $r_t > 1$: action becomes more likely
- $r_t < 1$: action becomes less likely

Step 2: PPO objective

$$L^{\text{PPO}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right]$$

What clipping does:

- Prevents probability ratio from changing too much
- Stops over-updating even if advantage is large

Full Training Objective

Overall loss function:

$$L = L_{\text{actor}} + \lambda_c L_{\text{critic}} + \lambda_m L_{\text{margin}} - \lambda_e H$$

This objective jointly optimizes:

- policy improvement (actor)
- value estimation (critic)
- ranking structure (margin)
- exploration (entropy)

Full Objective + Optimization

Total loss:

$$L = L_{\text{actor}} + \lambda_c L_{\text{critic}} + \lambda_m L_{\text{margin}} - \lambda_e H$$

1. Actor loss

$$L_{\text{actor}} = -\mathbb{E} \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right]$$

2. Critic loss

$$L_{\text{critic}} = \mathbb{E} \left[\left(V_{\psi}(s_t) - (r_t + \gamma V_{\psi}(s_{t+1})) \right)^2 \right]$$

3. Margin loss

$$L_{\text{margin}} = \mathbb{E} [\max(0, \Delta - (s_i - s_j))]$$

4. Entropy bonus

$$H = -\mathbb{E} \left[\sum_a \pi_{\theta}(a | s_t) \log \pi_{\theta}(a | s_t) \right]$$

Deadline Scheduling (RMAB Setup)

Problem motivation: EV charging stations

- A charging station has N charging slots (arms)
- At each round, only M vehicles can be charged (active arms)

State of each arm (vehicle):

$$s = (D, B)$$

- D : remaining time until deadline (max 12)
- B : remaining uncharged energy demand (max 9)

System Dynamics (EV Charging RMAB)

If an arm is ACTIVATED (charged):

- It receives 1 unit of service (or fractionally reduced demand)
- Demand reduces:

$$B \leftarrow \max(B - 1, 0)$$

- Immediate reward:

$$(1 - c)$$

per unit of charge delivered

If an arm is NOT activated (idle):

- No service is given $\rightarrow$ no immediate reward
- Deadline decreases:

$$D \leftarrow D - 1$$

When deadline expires ($D = 0$):

- Vehicle leaves the system
- Remaining demand B is unmet
- Station pays penalty: $F(B)$

Why this is an RMAB

Restless Multi-Armed Bandit structure:

- Each vehicle = one arm with Markov state (D, B)
- At each round, only M arms can be activated
- All arms evolve over time, whether activated or not (“restless”)

Key properties:

- Activated arms: gain reward + reduce demand
- Passive arms: age (deadline decreases)
- Final penalty couples time and remaining demand

Objective:

$$\max \mathbb{E} \left[\sum (1 - c) \cdot \text{service} - F(B_{\text{exit}}) \right]$$

Results

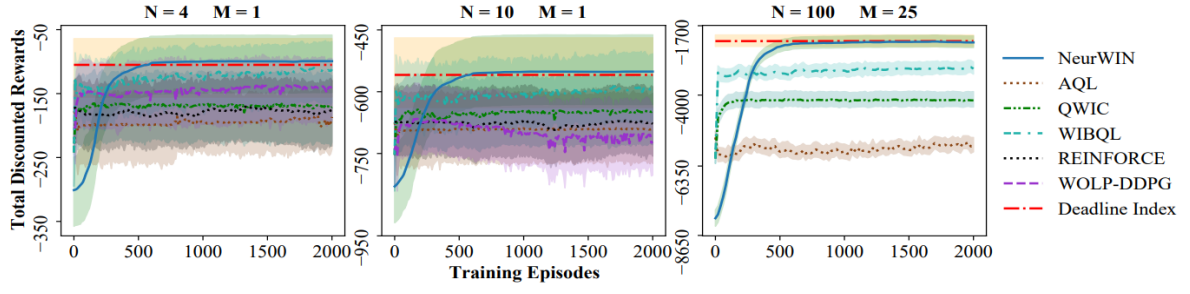


Figure 1: NeurWIN results

Results

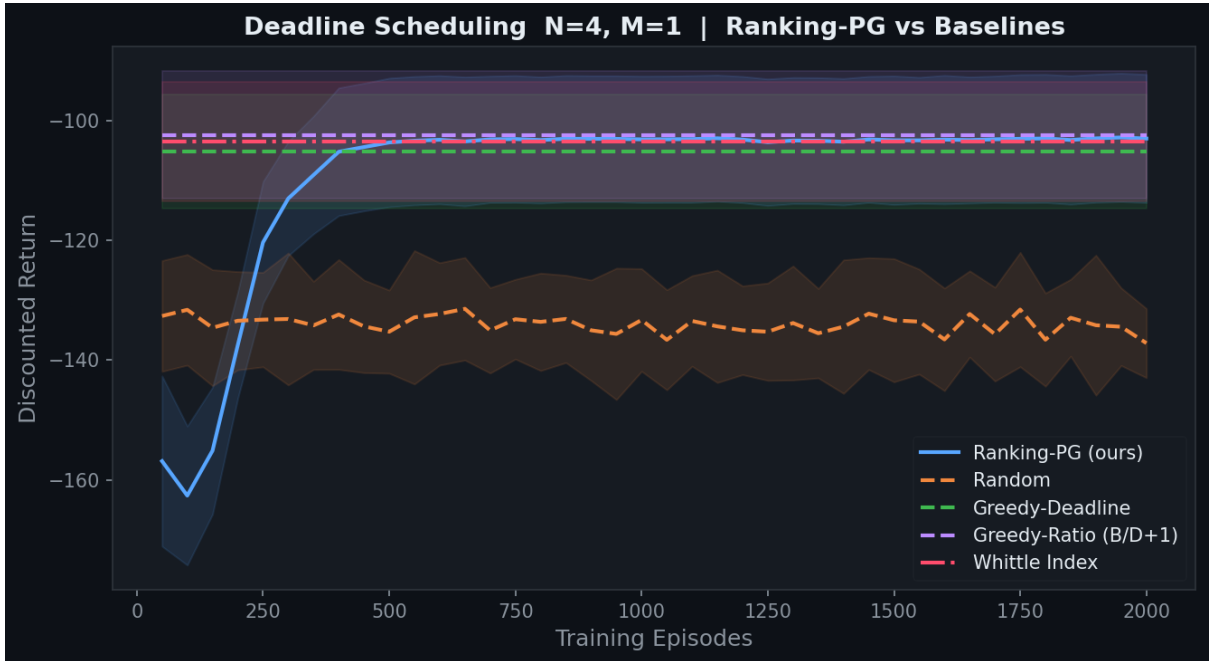


Figure 2: $N=4, M=1$

Results

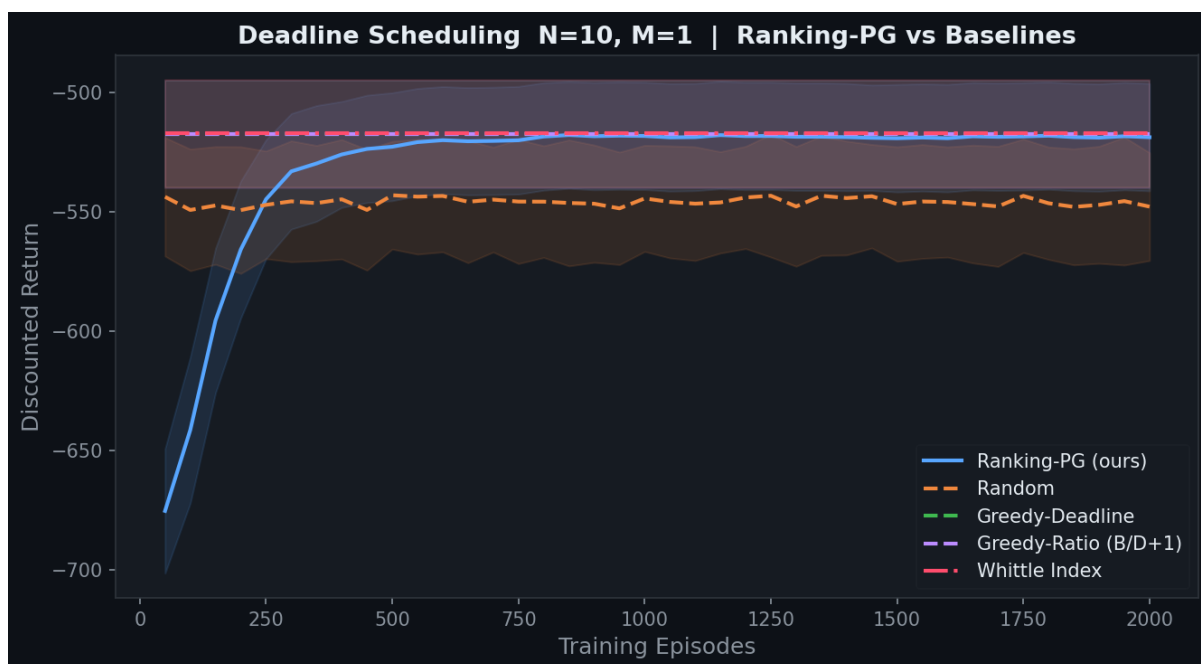


Figure 3: $N=10, M=1$

Results

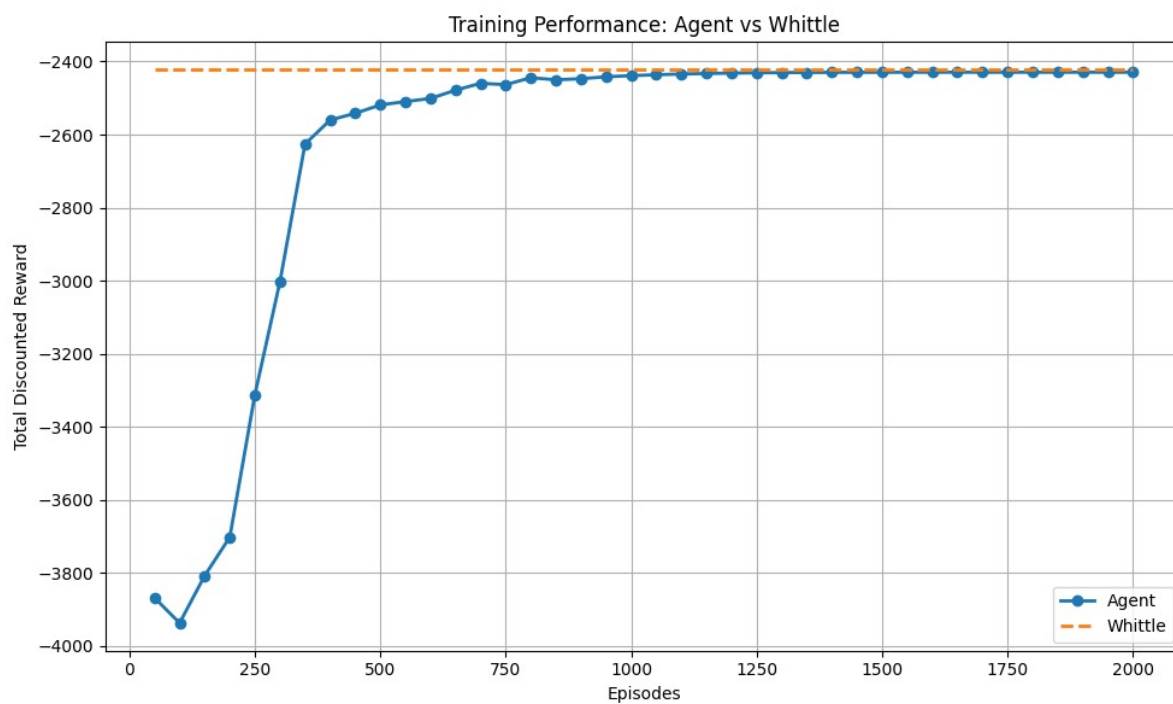


Figure 4: $N=100, M=25$

Do We Really Need PPO?

Experiment:

- Removed PPO clipping objective and trained using vanilla policy gradient.

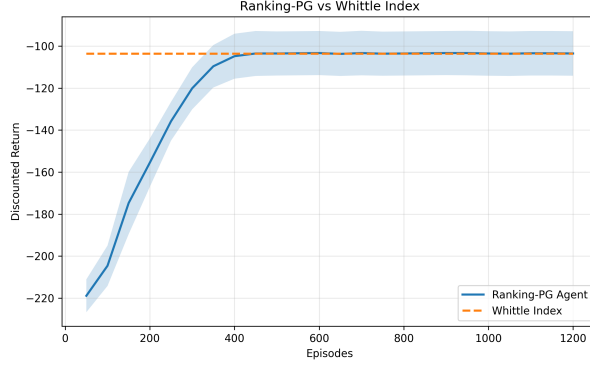


Figure 5: $N=4, M=1$

Observation: Convergence behavior was nearly unchanged. No major instability observed.

Machine Repair RMAB

Goal:

- We have N machines operating simultaneously
- Only M machines can be repaired at each timestep
- Need to decide *which machines to repair*

Machine state

Each machine has a discrete health state

$$s \in \{0, 1, \dots, S-1\}$$

- $s = 0$: perfectly healthy
- $s = S-1$: completely broken

Reward function

Healthy machines produce higher reward:

$$r(s) = (S-1) - s$$

Hence,

$$r(0) = S-1, \quad r(S-1) = 0$$

Constraint: At every timestep,

$$\sum_{i=1}^N a_i \leq M$$

where

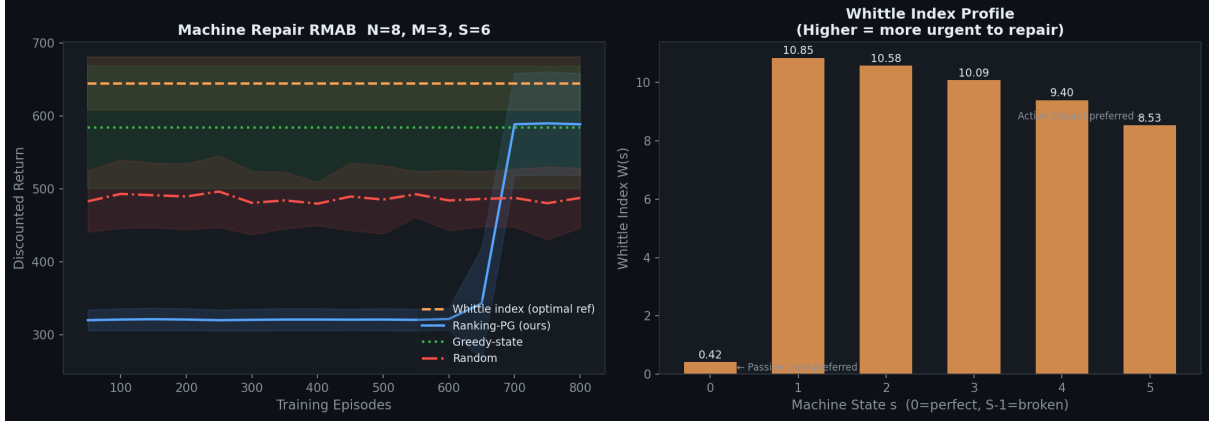
$$a_i = \begin{cases} 1 & \text{repair machine } i \\ 0 & \text{leave machine running} \end{cases}$$

Objective

Learn a scheduling policy maximizing long-term discounted reward:

$$\max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \right]$$

Results



Observations

- Ranking-PG substantially outperforms random scheduling
- Learned policy converges close to heuristic greedy scheduling
- Whittle policy remains best-performing reference
- Learned policy exhibits delayed performance jump, indicating late discovery of better ranking structure
- Whittle indices are not monotonic with degradation level:

$$W(1) > W(2) > W(3) > W(4) > W(5)$$

This suggests the optimal strategy prioritizes preventive maintenance rather than repairing already-failed machines. Hence the RMAB policy is genuinely nontrivial.

Reference Paper

Published as a conference paper at ICLR 2025

DOPL: DIRECT ONLINE PREFERENCE LEARNING FOR RESTLESS BANDITS WITH PREFERENCE FEEDBACK

Guojun Xiong^{1*}, Ujwal Dinesha², Debajoy Mukherjee², Jian Li³, Srinivas Shakkottai²

¹Harvard University, ²Texas A&M University, ³Stony Brook University

ABSTRACT

Restless multi-armed bandits (RMAB) has been widely used to model constrained sequential decision making problems, where the state of each restless arm evolves according to a Markov chain and each state transition generates a scalar reward. However, the success of RMAB crucially relies on the availability and quality of reward signals. Unfortunately, specifying an exact reward function in practice can be challenging and even infeasible. In this paper, we introduce PREF-RMAB, a new RMAB model in the presence of *preference* signals, where the decision maker only observes pairwise preference feedback rather than scalar reward from the activated arms at each decision epoch. Preference feedback, however, arguably

Problem: Preference-Based RMAB

- **Standard RMAB:** DM activates B arms, observes state transitions, and receives **scalar rewards** $r \in [0, 1]$.
- **The Constraint (PREF-RMAB):** In human-centric or noisy environments, scalar rewards are unobservable/unreliable.
- **Feedback:** We only observe **pairwise comparisons** (preferences) $\alpha \in \{0, 1\}$ between activated arms.
- **Goal:** Minimize regret against the optimal Oracle without ever seeing a true reward signal.

The Challenge

Traditional indices (like Whittle) rely on cardinal reward values. How do we rank arms using only ordinal feedback?

The DOPL Algorithm

DOPL operates in K episodes, each with H decision epochs:

1. **Interaction:** Activate B arms using current **Direct Index** $\mathcal{I}_n^k(s)$. Observe transitions and collect pairwise preference feedback.
2. **Estimation:** Update transition kernels $\hat{P}$ and empirical preference matrix $\hat{F}$.
3. **Inference:** Use Bradley-Terry model to infer preferences for un-dued pairs via a **Reference Arm** s_* .
4. **Optimization:** Solve the **Extended Linear Program (ELP)** with optimism bonuses to find occupancy measures ω^* .
5. **Projection:** Compute the new index $\mathcal{I}_n^{k+1}(s)$ directly from ω^* .

The Optimization Core: Extended LP

We find optimal occupancy measures ω by maximizing inferred preferences:

$$\max_{\omega} \sum_{n \in \mathcal{N}} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \omega_n(s, a) \cdot [\hat{Q}_n^k(s) + \text{Bonus}_n^k(s)]$$

Constraints:

- **Budget:** $\sum_{n,s} \omega_n(s, 1) = B$ (Average active budget).
- **Flow Conservation:** $\sum_a \omega_n(s', a) = \sum_{s,a} \omega_n(s, a) \hat{P}_n(s'|s, a)$.

$$\text{Direct Index Projection: } \mathcal{I}_n^k(s) = \frac{\omega_n^*(s, 1)}{\sum_a \omega_n^*(s, a)}$$

Theoretical Bridge: The Bradley-Terry Model

We assume the relationship between unobserved rewards r and observed preferences α follows the **Bradley-Terry (BT) model**:

$$P(\text{Arm } i \succ \text{Arm } j) = \frac{\exp(r_i)}{\exp(r_i) + \exp(r_j)}$$

- **Log-Odds:** We define preference-reference terms $Q_n(s) = r_n(s) - r_*(s_*)$.
- **Novelty:** DOPL avoids the "Indirect" trap. Instead of MLE-estimating absolute r , we learn the Q terms to rank arms relative to a baseline.
- This makes the algorithm invariant to global reward shifts (Scale-Invariant).

Results

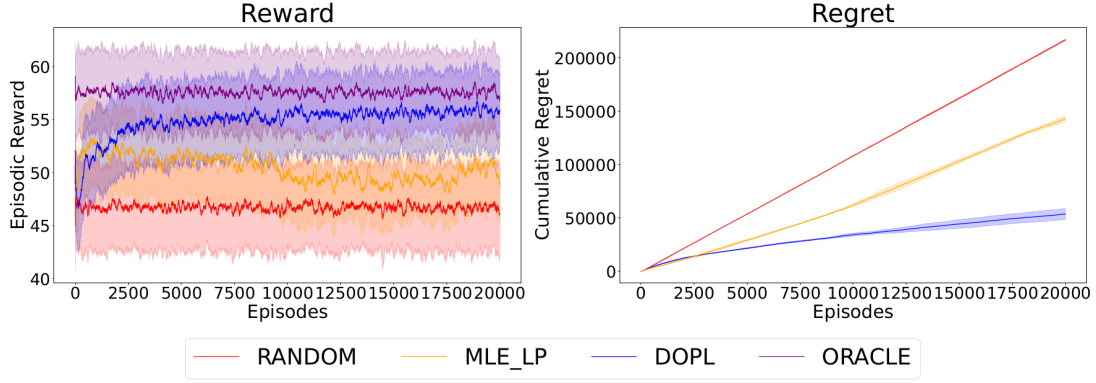


Figure 6: Comparison between DOPL and baseline algorithms

Novel Contribution: Evaluating Reward-Driven Preferences

The Experiment:

- Implemented a *Self-Labeling Oracle* where preferences are generated via $\alpha_{ij} = \mathbb{I}(R_i > R_j)$ using environment rewards.
- **Result:** Performance was comparable but **not superior** to direct reward-based algorithms.

Why it didn't exceed existing baselines:

- **Information Quantization:** Converting 32-bit scalar rewards to 1-bit binary feedback discards the *magnitude* of the reward gap.
- **Exploration Bottleneck:** Preference-based DOPL ignores reward outliers. Scalar-DOPL is penalized by high-variance reward signals, leading to "jittery" index policies..
- **Sample Efficiency:** Preference-based models require more samples to reach the same statistical confidence as cardinal rewards.

Results

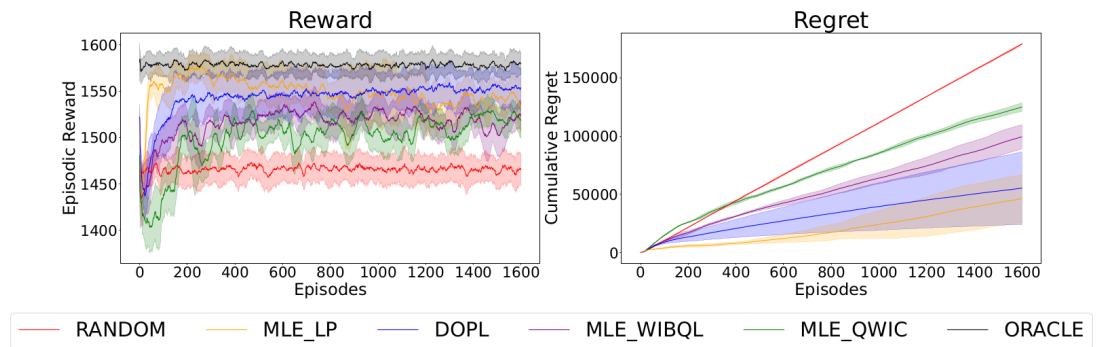


Figure 7: Regret Comparison: Reward-based vs. Preference-based DOPL.

Monte Carlo Corrected Q Learning

Instead of trusting only TD estimates, combine:

- Bootstrapped Q-values
- Monte Carlo returns

Monte Carlo Return:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

MC-Corrected Q-value:

$$\tilde{Q}_i(s, a) = \lambda_i(s, a) \hat{G}_i(s, a) + (1 - \lambda_i(s, a)) Q_i(s, a)$$

where:

$$\lambda_i(s, a) = \frac{n_i(s, a)}{n_i(s, a) + N_0}$$

Baseline Methods

QGI (Rested Bandits)

$$Q_i(s, a) \leftarrow Q_i(s, a) + \alpha [r + \gamma \max_{a'} Q_i(s', a') - Q_i(s, a)]$$

$$GI_i(s) = \frac{Q_i(s, 1) - Q_i(s, 0)}{1 - \gamma}$$

QWI (Restless Bandits)

$$Q_i(s, a) \leftarrow Q_i(s, a) + \alpha [r^{eff} + \gamma \max_{a'} Q_i(s', a') - Q_i(s, a)]$$

$$WI_i(s) = Q_i(s, 1) - Q_i(s, 0)$$

Our Method: MC-QGI / MC-QWI

MC-QGI Update

$$Q_i(s, a) \leftarrow Q_i(s, a) + \alpha [r + \gamma \max_{a'} \tilde{Q}_i(s', a') - Q_i(s, a)]$$

$$MCGI_i(s) = \frac{\tilde{Q}_i(s, 1) - \tilde{Q}_i(s, 0)}{1 - \gamma}$$

MC-QWI Update

$$Q_i(s, a) \leftarrow Q_i(s, a) + \alpha [r^{eff} + \gamma \max_{a'} \tilde{Q}_i(s', a') - Q_i(s, a)]$$

$$MCWI_i(s) = \tilde{Q}_i(s, 1) - \tilde{Q}_i(s, 0)$$

Subsidy μ (QWI) and Retirement M (QGI)

QWI — subsidy

Diagonal MC-corrected gap (double- Q average at $s=x$):

$$\Delta_i(x) = \frac{1}{2} \left[\tilde{Q}_i^A(x, x, 1) - \tilde{Q}_i^A(x, x, 0) + \tilde{Q}_i^B(x, x, 1) - \tilde{Q}_i^B(x, x, 0) \right].$$

$$\mu_i(x) \leftarrow \text{clip}(\mu_i(x) + \beta \Delta_i(x)).$$

Base reward in the augmented update: $b_i(x, a) = (1 - a) \mu_i(x) + a r_i$.

QGI — retirement

For active value $Q_i^x(s, 1)$ at reference x , diagonal $s=x$:

$$M_i(x) \leftarrow \text{clip}(M_i(x) + \beta (Q_i^x(x, 1) - M_i(x))).$$

Algorithm Flow

For each episode:

1. **Explore Arms:** Use behavior policy π_β .
2. **Collect Returns:** Compute Monte Carlo returns:

$$G_t$$

3. **Update MC-Corrected Q:** Compute:

$$\tilde{Q}$$

Update Q-values.

4. **Compute Indices**

Compute:

$$MCGI(s) \quad \text{or} \quad MCWI(s)$$

5. **Rank Arms:** Sort indices.
6. **Pull Top-K Arms:** Budget-constrained activation.
7. **Update Policy**

$$\pi_\beta \leftarrow (1 - \alpha) \pi_\beta + \alpha \pi_{greedy}$$

Novel Contributions

- First MC-corrected index learning for QWI/QGI.
- Combines:
 - Fast TD convergence
 - Unbiased MC estimates
- Better arm ranking during early exploration.
- Applicable to:
 - Rested bandits
 - Restless bandits

Evaluation Metrics

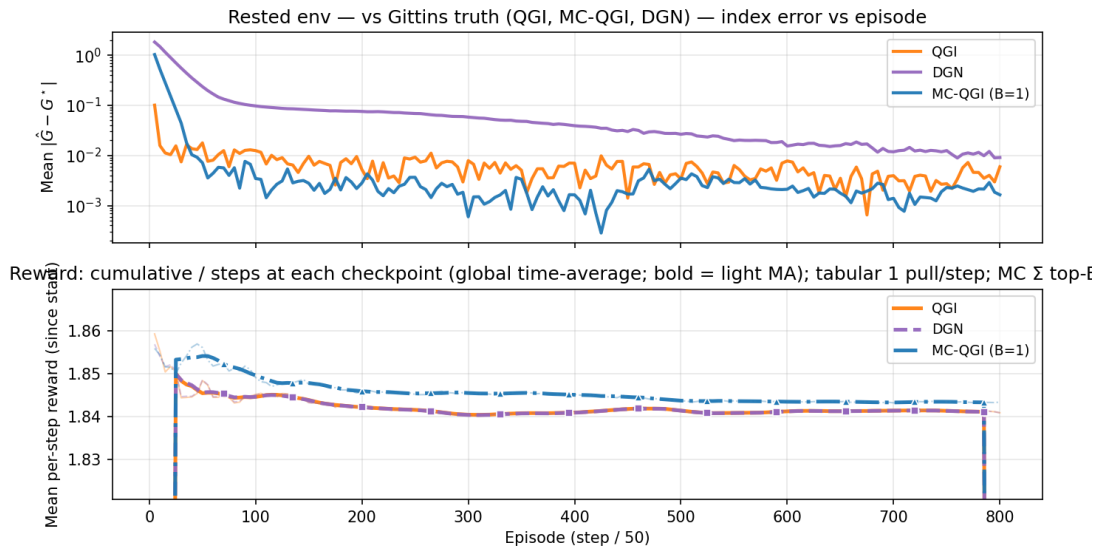
Compared against:

- QWI
- QGI
- QWINN
- DGN

Measured:

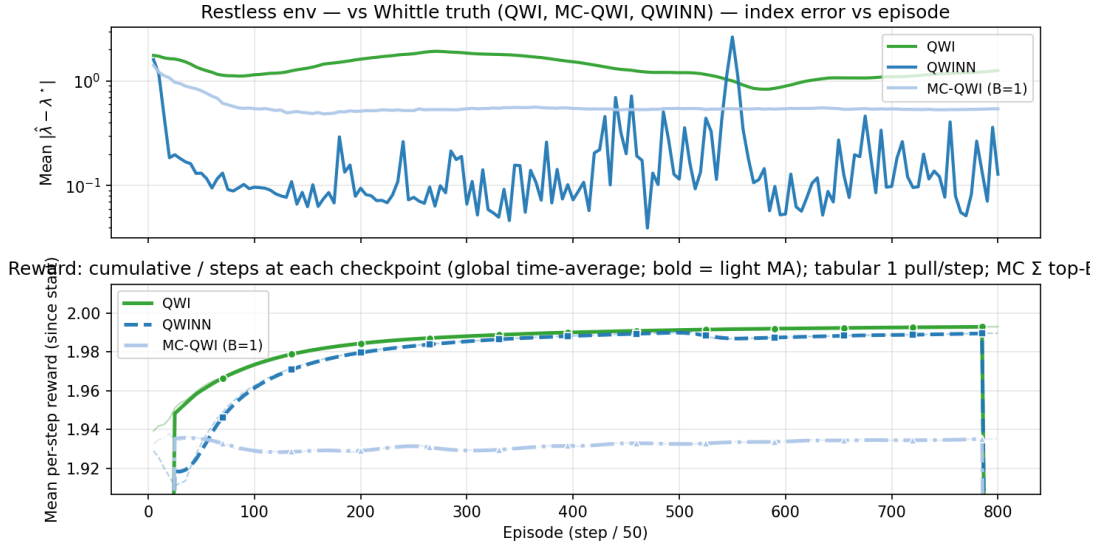
- Final average reward
- Convergence speed
- Variance across seeds
- Compute per episode

Results: rested vs Gittins (index error & reward)



K=5 arms N=5 states/arm $\gamma=0.9$ steps=40000 log every=250 seed=42 | mc episode len=50 smooth $w=1$ track reward=global_avg
MC: top-B concurrent activations per step, $B \in \{1\}$ (same K,N as tabular; tabular uses 1 arm/step).
env: RMABEnvironment.make_toy — rested track vs Gittins; restless track vs Whittle (same P,R, γ per K,N).

Results: restless vs Whittle (index error & reward)



Introduction to Threshold Policies

The Inefficiency of Generic RL in Monotone Environments

- Standard Deep RL algorithms (like DDPG or TD3) map states to actions via a generalized neural network $\pi_\theta(s)$.
- They treat every state independently. If the temperature is 75° and the AC turns on, standard RL does not inherently "know" that at 76° the AC should also be on.
- **The Monotone Property:**
 - In many control systems (inventory, scheduling, smart grids), optimal actions monotonically depend on a scalar trigger.
 - If action $a = 1$ is optimal at state λ , it remains optimal for all $\lambda' > \lambda$ (or $\lambda' < \lambda$).
- **The Authors' Goal:** Hardcode this inductive bias into the RL architecture to massively reduce the exploration space.

MDP Formulation & State Decoupling

Rigorous MDP Formulation

- The environment is an MDP $\mathcal{E} = (\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathbb{P}, \gamma)$.
- **Action Space:** Binary, $\mathcal{A} := \{0, 1\}$.
- **State Space Decoupling:** $s_t \in \mathcal{S}$ is factored into $s_t = (\lambda_t, v_t)$.
 1. **Scalar state** $\lambda_t \in \mathbb{R}$: The continuous trigger variable (e.g., current price, current inventory).
 2. **Vector state** $v_t \in \mathcal{V}$: The discrete contextual variables (e.g., time to deadline, seasonality).
- **Threshold Function:** The policy learns a mapping $\mu : \mathcal{V} \rightarrow \mathbb{R}$.
- **Action Execution:** $a_t = \mathbb{I}(\mu(v_t) > \lambda_t)$.

The Continuous Objective Function

- Let $\mu^\phi(v)$ be our threshold function parameterized by weights ϕ .
- Let $Q_{\mu^\phi}(\lambda, v, a)$ be the expected discounted reward from taking action a in state (λ, v) and following μ^ϕ thereafter.
- We want to find ϕ that maximizes the expected reward across all possible states.
- The global objective function integrates over the entire bounded continuous space $[-M, +M]$ of the scalar state λ :

$$K(\mu^\phi) := \int_{\lambda=-M}^{+M} \sum_{v \in \mathcal{V}} Q_{\mu^\phi}(\lambda, v, \mathbb{I}(\mu^\phi(v) > \lambda)) d\lambda \quad (1)$$

The Threshold Policy Gradient Theorem

The Mathematical Bottleneck: Bypassing the Integral

- We need to compute $\nabla_\phi K(\mu^\phi)$.
- The indicator function $\mathbb{I}(\mu^\phi(v) > \lambda)$ creates discontinuous jumps in the action space.
- **The Authors' Derivation Strategy:**
 1. Use the Fubini-Tonelli theorem to swap summation and integration.
 2. Break the integral into discrete intervals bounded by the thresholds of different vector states.
 3. Apply the **Leibniz Integral Rule** to differentiate under the integral sign.

The Threshold Policy Gradient Theorem

Theorem 1 (Gradient Expression)

If all vector states $v \in \mathcal{V}$ have distinct values of $\mu^\phi(v)$, the gradient of the objective function simplifies completely to:

$$\nabla_\phi K(\mu^\phi) \propto \sum_{v \in \mathcal{V}} \bar{\rho}(\mu^\phi(v), v) \underbrace{\left[Q_{\mu^\phi}(\mu^\phi(v), v, 1) - Q_{\mu^\phi}(\mu^\phi(v), v, 0) \right]}_{\text{Advantage at the Boundary}} \nabla_\phi \mu^\phi(v)$$

- $\bar{\rho}(\cdot)$ is the discounted state visitation distribution.
- The integral disappears entirely!

RMABs and the Whittle Index

The Curse of Restless Multi-Armed Bandits (RMABs)

- An RMAB system has N independent Markovian arms.
- In every timestep, the agent must activate exactly V arms out of N .
- **The Curse of Dimensionality:** The state space grows exponentially as $\mathcal{S}_1 \times \mathcal{S}_2 \times \dots \times \mathcal{S}_N$.
- **The Standard Heuristic:** The Whittle Index Policy.
 - Assign an independent "index" $W_i(s_{i,t})$ to each arm based on its current state.
 - Sort the arms by their indices.
 - Activate the top V arms.

Lagrangian Relaxation: Step 1 (Constraints)

Whittle's 1988 insight involves two stacked relaxations:

1. Relaxation 1: Hard $\rightarrow$ Average Constraint

$$\sum_i a_{i,t} = V \forall t \quad \longrightarrow \quad \mathbb{E} \left[\sum_i a_{i,t} \right] = V$$

2. Effect on Feasibility:

- The original feasible set is a strict subset of the relaxed one.
- Allows policies that activate $V + k$ arms now and $V - k$ later.
- This "time-averaging" is the first step toward decoupling the arms.

Lagrangian Relaxation: Step 2 (Decoupling)

2. Relaxation 2: Constrained $\rightarrow$ Lagrangian Dual

Introduce multiplier λ (activation tax):

$$\mathcal{L}(\pi, \lambda) = \mathbb{E} \left[\sum_t \gamma^t \sum_i (R_i - \lambda a_i) \right] + \frac{\lambda V}{1 - \gamma}$$

Why this decouples:

- Lagrangian is **additive** across dynamically independent arms.
- 1 PSPACE-hard problem becomes N independent problems:

$$\max_{\pi} \mathcal{L}(\pi, \lambda) = \sum_i \max_{\pi_i} (\text{Single-arm value with cost } \lambda)$$

- **Optimality:** Whittle is provably optimal in the limit as $N \rightarrow \infty$.

Reframing the Whittle Index as a Threshold Policy

- The Whittle index can be computed by looking at a **single arm in isolation** under an alternative problem formulation.
- **The Alternative Problem:**
 - The agent decides whether to activate arm i .
 - If activated, the agent pays an artificial *activation cost* λ .
 - Net reward: $r_{i,t} - \lambda a_{i,t}$.
- A threshold policy activates the arm iff $\mu_i(s_{i,t}) > \lambda$.
- **The Insight:** The value $\mu_i(s_{i,t})$ represents the maximum cost the agent is willing to pay. *This threshold is exactly the Whittle index $W_i(s_i)$.*

Breaking the Architectural Ceiling

The Architectural Ceiling of DeepTOP

When does the DeepTOP/Whittle framework fail? It is asymptotically optimal only if $N \rightarrow \infty$ and arms are strictly indexable. At finite N , three gaps emerge:

1. **Constraint Gap:** The Lagrangian relaxation enforces the budget on average, not per-timestep.
2. **Projection Gap:** Forcing a hard top- V selection at deployment is a heuristic that ignores multi-arm competition.

The Core Issue: DeepTOP relies entirely on *isolated, per-arm scoring*. An arm computes its threshold completely blind to the states of the other $N - 1$ arms competing for the budget.

JSA-DeepTOP: Joint-Summary-Augmented DeepTOP

[shrink=15]Algorithm 1: JSA-DeepTOP — Core Idea **Problem with DeepTOP:** Each arm computes its threshold *completely blind* to what all other $N-1$ arms are doing right now.

Fix: Add one extra input to each per-arm actor — the empirical state distribution $\hat{\mu}_t$ — a compact summary of the whole system’s current state:

$$\text{DeepTOP: } \mu_i^{\phi_i}(s_i) \longrightarrow \text{JSA-DeepTOP: } \mu_i^{\phi_i}(s_i, \underbrace{\hat{\mu}_t}_{\text{joint context}})$$

What this enables: An arm can reason — “*Most other arms are also in high-value states right now. Budget is competitive. I should bid higher than usual.*” DeepTOP cannot do this.

Why $\hat{\mu}_t$ specifically?

- It is the **sufficient statistic** for all symmetric functions of the joint state (de Finetti theorem).
- Its size is always $|S|$ — **fixed regardless of N** . The same network works at $N=10$ and $N=10,000$.

Technical Background: What is $\hat{\mu}_t$ and the Mean Field?

$\hat{\mu}_t$ is a **histogram** of how many arms are in each state:

$$\hat{\mu}_t(s) = \frac{1}{N} \sum_{j=1}^N \mathbb{1}[s_{j,t} = s]$$

The mean-field limit: As $N \rightarrow \infty$, the randomness in $\hat{\mu}_t$ vanishes. It converges to a *deterministic* trajectory $\bar{\mu}_t$ called the **mean field**. Whittle’s framework implicitly operates at this infinite-arm limit — it assumes the system always looks like its average.

At finite N : $\hat{\mu}_t$ fluctuates around $\bar{\mu}_t$ at rate $\mathcal{O}(1/\sqrt{N})$ (CLT). These fluctuations carry information Whittle discards.

Concrete example ($N = 5$, $|S| = 3$ states: Low/Mid/High):

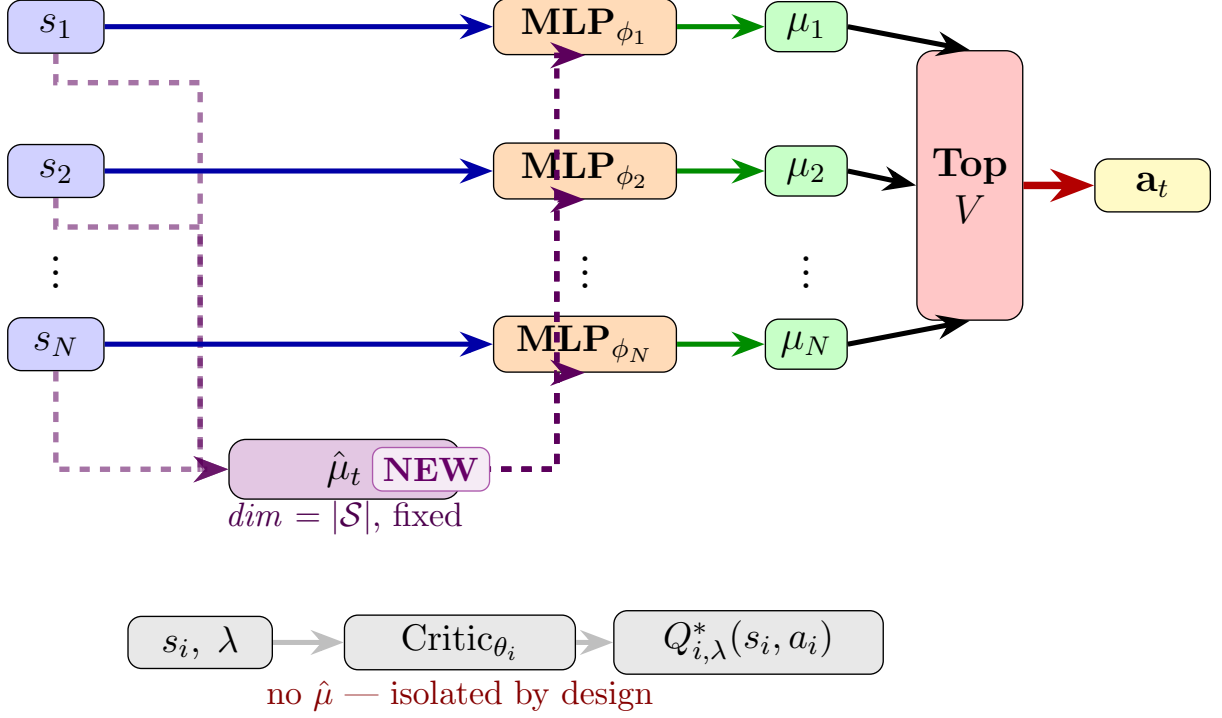
Arm	State
1	High
2	Mid
3	High
4	Low
5	High

$$\hat{\mu}_t = [0.2, 0.2, 0.6] \Rightarrow 60\% \text{ of arms are high.}$$

Signal: Budget is heavily contested. Every arm should raise its bid threshold.

Whittle sees: $\bar{\mu} = [0.33, 0.33, 0.33]$ (long-run avg). Misses the current spike entirely.

[shrink=8]JSA-DeepTOP: Architecture



■ Arm state $\dashrightarrow \hat{\mu}_t$ joint summary (new path) ■ Per-arm actor MLP ■ Critic (isolated)
[shrink=12]JSA-DeepTOP: Training Algorithm

Algorithm 1 JSA-DeepTOP

Require: N arms, budget V , per-arm actor networks $\{\phi_i\}$, per-arm critics $\{\theta_i\}$

- 1: Init replay buffer $\mathcal{D} \leftarrow \emptyset$
 - 2: **for** each timestep t **do**
 - 3: Observe joint state $\mathbf{s}_t = (s_1, \dots, s_N)$
 - 4: **Compute joint summary:** $\hat{\mu}_t(s) \leftarrow \frac{1}{N} \sum_i \mathbb{I}[s_{i,t} = s]$ *// $|\mathcal{S}|$ -dim histogram*
 - 5: **for** each arm i **do**
 - 6: $\mu_i \leftarrow \text{Actor}_{\phi_i}(s_i, \hat{\mu}_t)$ *// per-arm forward pass; $\hat{\mu}_t$ is the joint context*
 - 7: **end for**
 - 8: $\mathbf{a}_t \leftarrow \text{top-}V(\{\mu_i\})$; execute; observe $\mathbf{r}_t, \mathbf{s}_{t+1}$
 - 9: Store $(\mathbf{s}_t, \hat{\mu}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$ in $\mathcal{D}$; sample minibatch $\mathcal{B}$
 - 10: **Critic update** (per arm, no $\hat{\mu}$ input):
 Minimise $\mathbb{E}_{\mathcal{B}}[(Q_{\theta_i}(s_i, a_i) - y_i)^2]$ for each i
 - 11: **Actor update** (augmented gradient theorem):
 $\nabla_{\phi_i} \propto \mathbb{E}_{\mathcal{B}}[\mathbb{E}_{\hat{\mu}}[A_i(s_i, \hat{\mu}) \cdot \nabla_{\phi_i} \mu_i(s_i, \hat{\mu})]]$
 $A_i = Q_{\theta_i}(\mu_i, 1) - Q_{\theta_i}(\mu_i, 0)$ *// advantage at threshold boundary*
 - 12: **end for**
-

Differences from DeepTOP: Line 4 (compute $\hat{\mu}_t$) and Line 6 (pass $\hat{\mu}_t$ to actor) are the *only* additions. Critic update is identical to DeepTOP.

JSA-DeepTOP: Why the Math Still Works

Concern: Adding $\hat{\mu}$ as input might break the clean gradient identity from Theorem 1.

Augmented Threshold Policy Gradient

$$\nabla_{\phi_i} K_i^{\text{aug}} = \mathbb{E}_{\hat{\mu}} \left[|\mathcal{S}| \sum_{s_i} \bar{\rho}(s_i, \hat{\mu}) \underbrace{[Q_i(\mu_i, 1) - Q_i(\mu_i, 0)]}_{\text{advantage at threshold}} \nabla_{\phi_i} \mu_i(s_i, \hat{\mu}) \right]$$

Proof in 3 moves:

1. **Condition on $\hat{\mu}$:** For fixed $\hat{\mu}$, the function $\mu_i(\cdot, \hat{\mu})$ is a regular per-arm threshold. Original Theorem 1 applies pointwise.
2. **Swap ∇ and $\mathbb{E}$:** Off-policy assumption + dominated convergence: $\nabla_{\phi_i} \mathbb{E}_{\hat{\mu}}[\cdot] = \mathbb{E}_{\hat{\mu}}[\nabla_{\phi_i} \cdot]$.
3. **Combine:** Plug step 1 inside step 2. Done.

Deeper point: Theorem 1 was always more general than DeepTOP’s paper claimed. μ can depend on *anything differentiable* — not just s_i .

JSA-DeepTOP: One Change, Clean Attribution

Exactly one architectural element changes.

Modified:

- Actor input: $(s_i) \rightarrow (s_i, \hat{\mu}_t)$
- Replay buffer stores $\hat{\mu}_t$ snapshots

Unchanged:

- Average-budget Lagrangian relaxation
- Per-arm critic $Q_{i,\lambda}^*$ (no joint critic)
- Hard Top-V at deployment
- If actor zeros out $\hat{\mu}$ weights $\Rightarrow$ exact DeepTOP

Expressivity Hierarchy (proved)

$$\mathcal{T}_{\text{DeepTOP}} \subsetneq \mathcal{T}_{\text{JSA}} \subsetneq \mathcal{T}_{\text{DeepTOP-J}}$$

JSA **strictly generalises** DeepTOP.

Scientific payoff: Any improvement in experiments can only come from joint-state awareness. There is no confounding.

Failure mode: On problems with no joint structure (e.g. recovering bandits), the actor learns to ignore $\hat{\mu}$ and $\text{JSA} \approx \text{DeepTOP}$. No harm done.

DeepTOP-J: Joint DeepTOP

Algorithm 2: DeepTOP-J — Core Idea

Limitation of JSA-DeepTOP: $\hat{\mu}_t$ is a histogram. It tells you *how many* arms are in each state, not *which* specific arm is in which state.

What JSA misses:

Arms 3 and 7 have different transition probabilities ($p_3 \gg p_7$). Both are in a high state. Who should be activated? $\hat{\mu}$ sees “two arms in high states” and averages them. A joint actor can see that arm 3 specifically has a high climb probability and prioritise it.

DeepTOP-J fix: Replace N separate per-arm MLPs with **one shared Set Transformer** that processes all arm states simultaneously:

$$(\mu_1, \dots, \mu_N) = \underbrace{\text{SetTransformer}_\phi}_{\text{one shared network, parameters } \phi} \left(\{(s_i, \xi_i)\}_{i=1}^N \right)$$

Self-attention within the transformer allows arm i to directly attend to arm j 's exact state — capturing asymmetric, pairwise dependencies that $\hat{\mu}$ (an average) cannot express.

All N thresholds from *one* forward pass.

[shrink=10]Technical Background: Permutation Equivariance & Self-Attention

Permutation equivariance:

The RMAB is *symmetric*. Swapping arm 3 and arm 7's labels doesn't change the problem — the actions should swap correspondingly.

Formally: $f(\sigma(\mathbf{s})) = \sigma(f(\mathbf{s}))$ for any permutation σ .

An MLP on concatenated states *breaks* this — it treats arm 1 differently from arm 7 by position in the vector. That's wrong.

Token metadata ξ_i : For heterogeneous arms, give each arm a metadata vector (e.g. $\xi_i = (p_i, q_i)$). Equivariance holds over (s_i, ξ_i) *pairs* — relabelling arms *with* their metadata leaves the problem unchanged.

Self-attention mechanism:

Each arm i computes *how much to attend to arm j* :

$$\alpha_{ij} = \text{softmax}\left(\frac{q_i^\top k_j}{\sqrt{d}}\right)$$

Then updates its representation:

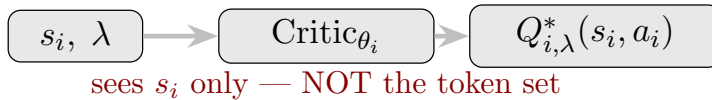
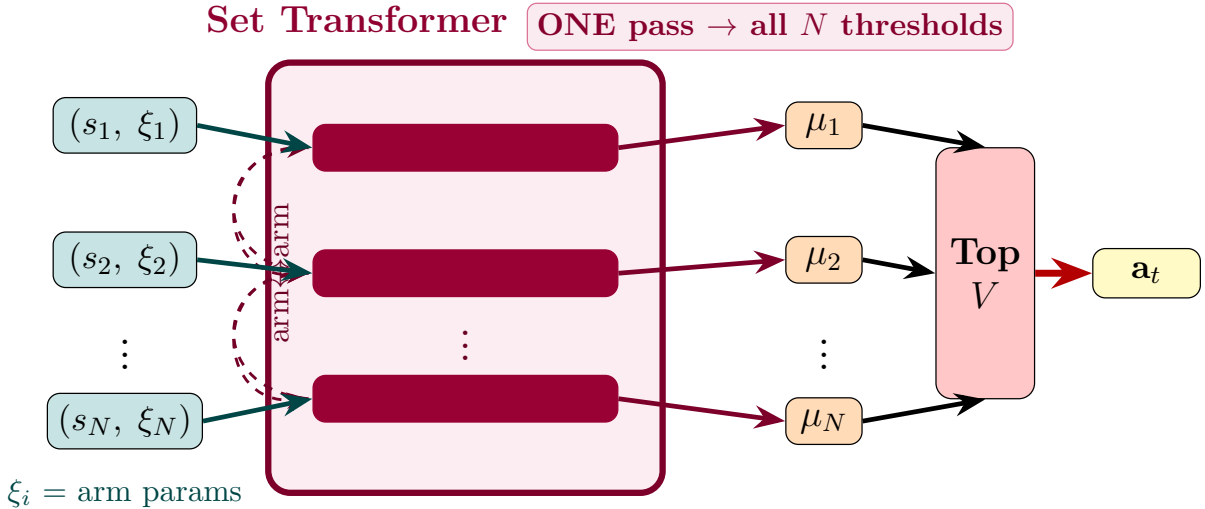
$$h_i \leftarrow h_i + \sum_j \alpha_{ij} v_j$$

where q_i, k_j, v_j are learned projections (queries, keys, values).

What this buys us: Arm i learns to *specifically* look at certain arms (high α_{ij}) and ignore others (low α_{ij}). $\hat{\mu}$ cannot do this — it weights all arms equally.

Equivariant by construction: softmax is symmetric in j .

[shrink=8]DeepTOP-J: Architecture



■ Token (s_i, ξ_i) $\dashrightarrow$ Cross-arm attention ■ Threshold μ_i ■ Critic (isolated)
[shrink=12]DeepTOP-J: Training Algorithm

Algorithm 2 DeepTOP-J

Require: N arms with metadata $\{\xi_i\}$, budget V , **shared** actor ϕ , per-arm critics $\{\theta_i\}$

```

1: Init replay buffer  $\mathcal{D} \leftarrow \emptyset$ 
2: for each timestep  $t$  do
3:   Observe joint state  $\mathbf{s}_t = (s_1, \dots, s_N)$ 
4:   Form token set:  $\mathbf{T}_t \leftarrow \{(s_i, \xi_i)\}_{i=1}^N$ 
5:   Joint forward pass (one call, all arms):
       $(\mu_1, \dots, \mu_N) \leftarrow \text{SetTransformer}_\phi(\mathbf{T}_t)$  // cross-arm attention inside
6:    $\mathbf{a}_t \leftarrow \text{top-}V(\{\mu_i\})$ ; execute; observe  $\mathbf{r}_t, \mathbf{s}_{t+1}$ 
7:   Store  $(\mathbf{T}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$  in  $\mathcal{D}$ ; sample minibatch  $\mathcal{B}$ 
8:   Critic update (per arm — no joint input):
      Minimise  $\mathbb{E}_{\mathcal{B}}[(Q_{\theta_i}(s_i, a_i) - y_i)^2]$  for each  $i$ 
9:   Shared actor update (joint gradient — all arms contribute):
       $\nabla_\phi \propto \mathbb{E}_{\mathcal{B}}\left[\sum_{i=1}^N A_i(s_i) \cdot \nabla_\phi \mu_i(\mathbf{T})\right]$ 
       $A_i = Q_{\theta_i}(\mu_i, 1) - Q_{\theta_i}(\mu_i, 0)$ 
10: end for

```

Two key differences from JSA: (1) Line 5: ONE transformer call replaces N separate MLP calls.
(2) Line 9: actor gradient *sums over all arms* — all N advantage signals train the *shared* ϕ .
[shrink=5]DeepTOP-J: Why Set Transformer Specifically?

Three candidate families:

Architecture	Equiv?	Pairwise?
MLP (concat)	×	×
Deep Sets	✓	×
Set Transformer	✓	✓

Deep Sets computes $\mu_i = h(s_i, \rho(\{g(s_j)\}))$ — aggregate-then-combine. This is mathematically equivalent to JSA-DeepTOP with a learned aggregator. It cannot express pairwise arm dependencies.
Set Transformer (Lee et al. 2019): stacked multi-head self-attention. **Universal approximator** of permutation-equivariant functions on $\mathcal{S}^N$.

What attention adds:

“Arm i ’s threshold should rise *because arm j is in state s_j* ” — a pairwise dependency. Deep Sets only sees the bag.

Proved (Proposition 1):

$$\mathcal{T}_{\text{per-arm}} \subsetneq \mathcal{T}_{\text{JSA}} \subsetneq \mathcal{T}_{\text{DeepTOP-J}}$$

All three containments are *strict*.

DeepTOP-J: The Asymmetric Actor-Critic Design

Actor is fully joint. Critic is strictly per-arm. This asymmetry is *theoretically required*.

Why the critic must stay per-arm:

The critic estimates $Q_{i,\lambda}^*(s_i, a_i)$ — the value of the **single-arm alternative** MDP $\mathcal{M}_i^\lambda$. $\mathcal{M}_i^\lambda$ is defined in isolation by construction (Whittle 1988). It has zero dependence on other arms.

A joint critic would:

- Conflate dynamic independence with decisional coupling
- Produce Q-values that don’t correspond to any valid single-arm MDP
- Suffer exponential state space $|\mathcal{S}|^N$ for no theoretical gain

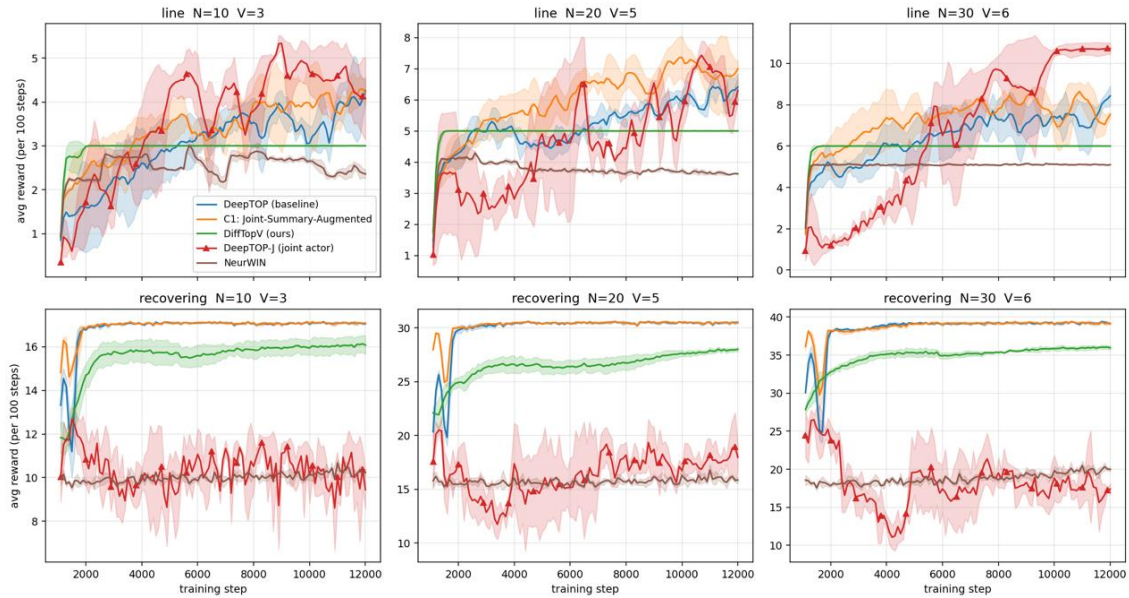
The Asymmetry Principle
Choosing thresholds = joint decision.
 $\Rightarrow$ Actor sees all arm states.

Evaluating a threshold = per-arm calculation (within the alternative MDP).
 $\Rightarrow$ Critic sees one arm only.

These are fundamentally different questions.

Practical payoff: Per-arm critics are cheap, data-efficient, and train with stable temporal-difference updates.

Experimental Results: Training Curves



Setup: 5 algorithms $\times$ 2 environments $\times$ 3 sizes ($N = 10, 20, 30$). Shaded bands = std across seeds. Top: line bandits (heterogeneous Markov chains, $p_i \in [0.2, 0.8]$). Bottom: recovering bandits (waiting-time arms). . On line bandits, our extensions help — but unevenly. On recovering bandits, the picture is essentially the opposite. The next three slides walk through this carefully.

Reading the Line-Bandit Results

Line bandits have substantial joint structure: heterogeneous arms (p_i spread over $[0.2, 0.8]$) competing to be near state 99 where reward peaks.

- **C1 / JSA-DeepTOP (orange):** Best at $N=10, 20$, slightly fades at $N=30$. Mean-field summary captures the most useful joint signal — *which arms are climbing well right now*.
- **DeepTOP-J (red):** Wide variance bands, but *best final performance at $N=30$* . Set Transformer needs more data to stabilize attention weights; once it does, it learns asymmetric arm-arm dependencies that beat mean-field.
- **NeurWIN (brown):** Lowest. Confirms that single-arm REINFORCE on the alternative MDP misses the joint adaptation DeepTOP captures from real trajectories.
- **DiffTopV (green):** Plateaus at V — exploratory third extension; details and analysis in appendix.

Reading the Recovering-Bandit Results

Recovering bandits is structurally different: each arm's reward depends only on *its own* waiting time. The problem is nearly decomposable.

- **C1 (orange) $\approx$ DeepTOP (blue):** Curves overlap almost perfectly across all N . Joint information adds *nothing* when the optimal policy is genuinely per-arm.
- **DeepTOP-J (red):** *Catastrophic regression* — falls all the way to NeurWIN’s level (single-arm baseline). Set Transformer’s extra capacity has no joint signal to fit, so it overfits to noise. Wide variance bands confirm training instability.
- **Implication:** More expressivity is not free. When joint structure is absent, joint architectures *actively hurt* via parameter inflation and gradient noise.
- **DiffTopV (green):** Slight underperformance; see appendix for details.

Synthesis: What the Experiments Taught Us

Algorithm	When it helps	When it hurts
JSA-DeepTOP	Joint structure exists, finite N	Decomposable problem (no harm; no gain)
DeepTOP-J	Strong joint structure + enough data	Decomposable; or insufficient training data

The $1/\sqrt{N}$ hypothesis (H1): *Partially supported.* Holds on line bandits where joint structure is real. Fails on recovering bandits where the gap was zero to begin with.

Hypothesis H2 (joint > mean-field): *Conditionally supported.* DeepTOP-J > JSA only on line bandits at $N=30$ — and only after Set Transformer training stabilizes.

Most defensible takeaway: JSA-DeepTOP is the safe extension. It either ties or beats baseline; never catastrophically loses. Pays a small architectural cost for variance-bounded improvement.

Conclusion

Theoretical contributions:

1. **JSA-DeepTOP:** Augmented threshold policy gradient theorem — DeepTOP’s framework extends to any differentiable joint-input parameterization.
2. **DeepTOP-J:** Strict expressivity hierarchy ($\mathcal{T}_{\text{per-arm}} \subsetneq \mathcal{T}_{\text{JSA}} \subsetneq \mathcal{T}_{\text{joint}}$) realized via Set Transformer.

Empirical findings (honest):

- JSA-DeepTOP: **robust** — wins or ties across all settings.
- DeepTOP-J: **environment-sensitive** — best on structured joint problems, fails on decomposable ones.

Future work: (i) adversarial benchmarks with explicit pairwise structure to stress-test H2; (ii) regularization for DeepTOP-J to mitigate over-parameterization on simple environments.

An exploratory third extension — DiffTopV, a differentiable soft top-V relaxation — is presented in the appendix.

Questions or Discussion?

Appendix: DiffTopV (Exploratory Third Extension)

Appendix — Algorithm 3: DiffTopV

The Problem: Baseline methods use a hard sorting operation to pick the top- V arms. Sorting is non-differentiable. Arms never learn how their score impacts the budget availability for others.

The DiffTopV Relaxation:

- We replace discrete selection with a *differentiable, soft top-V layer* via an implicit Lagrangian threshold $\lambda^*(f)$:

$$\sum_{i=1}^N \sigma(\beta(f_i - \lambda^*(f))) = V$$

- **Exact Enforcement:** Unlike standard softmax, this equation guarantees the sum of probabilities exactly equals V per timestep.
- Recovers hard top- V exactly as temperature $\beta \rightarrow \infty$.

Appendix — DiffTopV: Why λ^* Always Exists and Is Unique

Concern: Solving an implicit equation per timestep could be expensive or unstable. Is $\lambda^*(f)$ even well-defined?

Theorem (Existence & Uniqueness)

For any scores $f \in \mathbb{R}^N$, budget $V \in (0, N)$, and $\beta > 0$, the equation $G(f, \lambda) = \sum_i \sigma(\beta(f_i - \lambda)) = V$ has a unique solution $\lambda^*(f) \in \mathbb{R}$.

Proof by intermediate value theorem:

- G is **strictly decreasing** in λ (each sigmoid term has derivative $-\beta\sigma' < 0$).
- As $\lambda \rightarrow -\infty$: $G \rightarrow N$. As $\lambda \rightarrow +\infty$: $G \rightarrow 0$.
- Since $0 < V < N$ and G is continuous and strictly monotone, a unique λ^* exists.

Practical consequence: Bisection converges in ~ 50 iterations to machine precision. At $N = 100$ this is ~ 5000 flops — negligible vs. one actor forward pass ($\sim 10^4$ – 10^5 flops).

Corollary (Exact Budget): $\sum_i p_i(f) = V$ *exactly*, every timestep, every $\beta > 0$.

Appendix — DiffTopV: The Competition Gradient

Using the Implicit Function Theorem on the root-finding bisection, we derive a closed-form policy gradient:

$$\frac{\partial p_j^\phi}{\partial f_i^{\phi_i}}(s) = \beta\sigma' \cdot \left[\underbrace{\delta_{ij}}_{\text{Direct Drive}} - \underbrace{\frac{\partial \lambda^*}{\partial f_i}(f)}_{\text{Competition Term}} \right]$$

- **Off-Diagonal Information:** Actor i 's gradient now explicitly incorporates the marginal advantages of *every other arm*.
- If arm i raises its score, it increases the global budget pressure (λ^*), directly suppressing the activation probability of arm j .

Empirical observation: On line bandits this gradient drives DiffTopV to a *lazy equilibrium* at exactly V reward — the competition term equalizes scores rather than discriminating between arms. This is the failure mode that motivates further work (β -annealing, score normalization).

Appendix — DiffTopV: Soft Training, Hard Deployment

A subtle but principled design choice:

	Training	Deployment
Action selection	Soft probabilities p_i	Hard top- V on scores f
Gradient flow	Through p_i via λ^*	N/A
Budget satisfied	Exactly (corollary above)	Exactly (by sorting)

Why hard at deployment?

- **Direct comparability:** DeepTOP, NeurWIN, Whittle all use hard top- V . Apples-to-apples.
- **Lower variance evaluation:** Stochastic activation introduces sampling noise.

Asymptotic Consistency

As $\beta \rightarrow \infty$, $\lambda^*(\beta) \rightarrow f_{(V)}$ (the V -th order statistic), and $p_i \rightarrow \mathbb{I}[i \in \text{TopV}(f)]$ pointwise. So the training objective converges to the deployment objective.